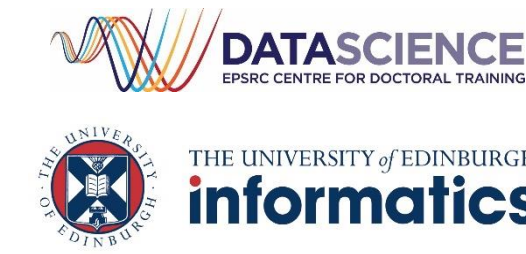


EvoGrad: Efficient Gradient-Based Meta-Learning and Hyperparameter Optimization

Ondrej Bohdal, Yongxin Yang,
Timothy Hospedales



Samsung Research



Overview

Efficient approach for gradient-based meta-learning

- Existing approaches: expensive second-order derivatives and a longer computational graph
- EvoGrad: first-order derivatives only

Inspired by evolutionary techniques to efficiently compute hypergradients
Significantly lower runtime and memory usage, with similar performance as existing methods

- Enables scaling meta-learning to larger network architectures

Evaluated on several recent meta-learning applications:

- Cross-domain few-shot learning with feature-wise transformations
- Noisy label learning with MetaWeightNet
- Low-resource cross-lingual learning with MetaXL

Background

Goal: estimate hyperparameters λ that minimize the validation loss ℓ_V of the model parameterized by θ and trained with loss ℓ_T and λ

$$\lambda^* = \arg \min_{\lambda} \ell_V^*(\lambda), \text{ where } \ell_V^*(\lambda) = \ell_V(\lambda, \theta^*(\lambda)) \text{ and } \theta^*(\lambda) = \arg \min_{\theta} \ell_T(\lambda, \theta)$$

Necessary to calculate hypergradient $\frac{\partial \ell_V^*(\lambda)}{\partial \lambda} = \frac{\partial \ell_V(\lambda, \theta^*(\lambda))}{\partial \lambda} + \frac{\partial \ell_V(\lambda, \theta^*(\lambda))}{\partial \theta^*(\lambda)} \frac{\partial \theta^*(\lambda)}{\partial \lambda}$

Direct term $\frac{\partial \ell_V(\lambda, \theta^*(\lambda))}{\partial \lambda}$ is typically zero, so simple first-order approximation is not available

EvoGrad update

Evolutionary inner step

- Sample K random perturbations ϵ and apply them to model parameters θ as $\theta_k = \theta + \epsilon_k$
- Compute training losses $\ell_k = f(\mathcal{D}_T | \theta_k, \lambda)$ for K models on the current minibatch $\mathcal{D}_T$
- Calculate weights for each model as $w_1, w_2, \dots, w_K = \text{softmax}([- \ell_1, - \ell_2, \dots, - \ell_K] / \tau)$

- Update model parameters via the affine combination

$$\theta^* = w_1 \theta_1 + w_2 \theta_2 + \dots + w_K \theta_K$$

Compute hypergradient using validation data $\frac{\partial \ell_V}{\partial \lambda} = \frac{\partial f(\mathcal{D}_V | \theta^*)}{\partial \lambda}$

- Inner loop does not use gradients, so overall the method is first-order

Do standard update of the base model afterwards

Comparison to existing methods

Table 1: Comparison of hypergradient approximations of $T_1 - T_2$ and EvoGrad.

Method	Hypergradient approximation
$T_1 - T_2$	$\frac{\partial \ell_V}{\partial \lambda} - \frac{\partial \ell_V}{\partial \theta} \times \mathbf{I} \frac{\partial^2 \ell_T}{\partial \theta \partial \lambda}$
EvoGrad (ours)	$\frac{\partial \ell_V}{\partial \lambda} + \frac{\partial \ell_V}{\partial \theta} \times \mathcal{E} \frac{\partial w}{\partial \ell} \frac{\partial \ell}{\partial \lambda} = \frac{\partial \ell_V}{\partial \lambda} + \frac{\partial \ell_V}{\partial \theta} \times \mathcal{E} \frac{\partial \text{softmax}(-\ell)}{\partial \lambda}$

Table 2: Comparison of asymptotic memory and operation requirements of EvoGrad and $T_1 - T_2$ meta-learning strategies. P is the number of model parameters, H is the number of hyperparameters. $K \ll H$ is the number of model copies in EvoGrad. Note this is a first-principles analysis, so the time requirements are different when using e.g. reverse-mode backpropagation that uses parallelization.

Method	Time requirements	Memory requirements
$T_1 - T_2$	$\mathcal{O}(PH)$	$\mathcal{O}(P + H)$
EvoGrad (ours)	$\mathcal{O}(KP + H)$	$\mathcal{O}(P + H)$

Graphical illustration

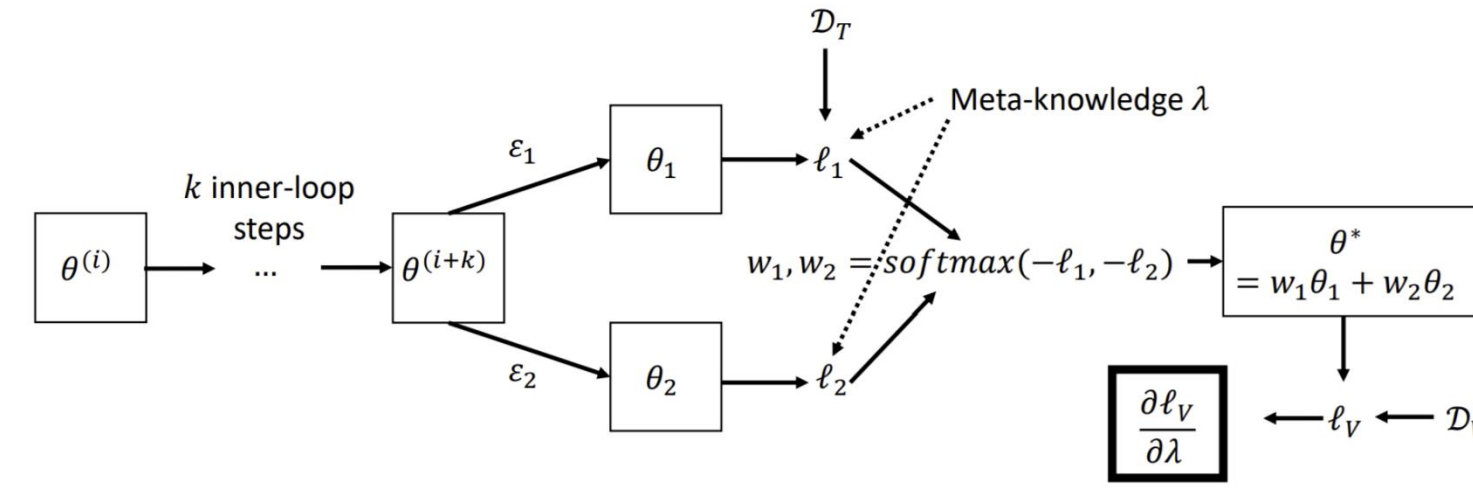


Figure 1: Graphical illustration of a single EvoGrad update using $K = 2$ model copies.

Illustration using a 1-dimensional problem

Minimize $f_V(x) = (x - 0.5)^2$ where parameter x is optimized using SGD with $f_T(x) = (x - 1)^2 + \lambda \|x\|_2^2$ that includes a meta-parameter λ
Closed-form solution for the hypergradient is $g(\lambda) = (\lambda - 1)/(\lambda + 1)^3$

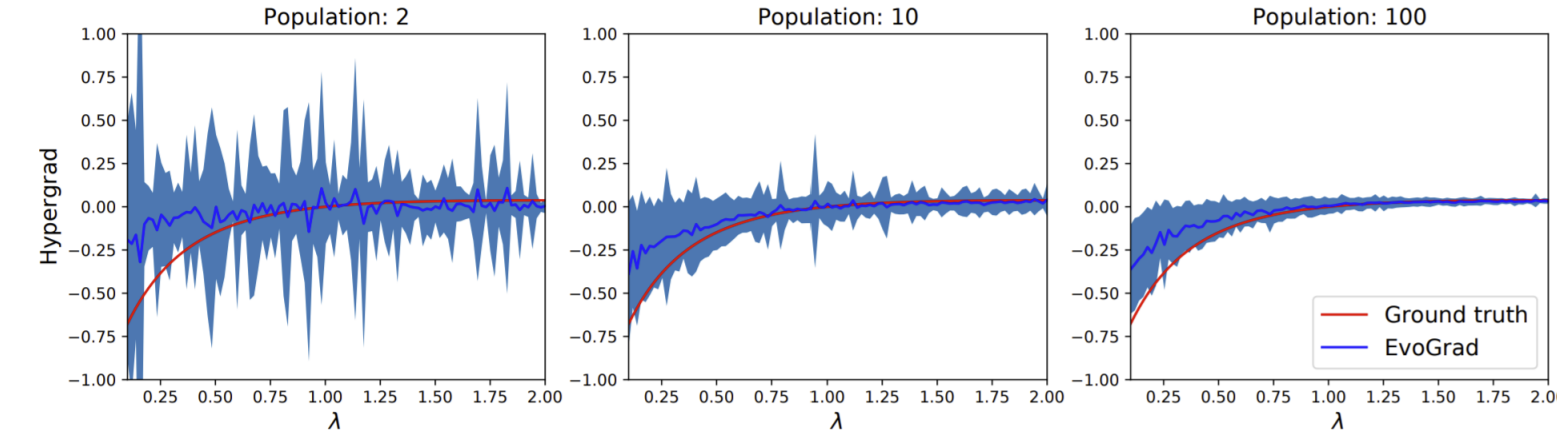


Figure 2: Comparison of the hypergradient $\partial f_V / \partial \lambda$ estimated by EvoGrad vs the ground-truth.

Cross-domain few-shot classification via learned feature-wise transformation

Table 3: RelationNet test accuracies (%) and 95% confidence intervals across test tasks on various unseen datasets. LFT EvoGrad can scale to ResNet34 on all tasks within 12GB GPU memory, while vanilla second-order LFT $T_1 - T_2$ cannot. We also report the results of our own rerun of the LFT approach using the official code – denoted as *our run*. EvoGrad can clearly match the accuracies obtained by the original approach that uses $T_1 - T_2$.

	Model	Approach	CUB	Cars	Places	Plantae
5-way 1-shot	ResNet10	-	44.33 $\pm$ 0.59	29.53 $\pm$ 0.45	47.76 $\pm$ 0.63	33.76 $\pm$ 0.52
		FT	44.67 $\pm$ 0.58	30.38 $\pm$ 0.47	48.40 $\pm$ 0.64	35.40 $\pm$ 0.53
		LFT $T_1 - T_2$	48.38 $\pm$ 0.63	32.21 $\pm$ 0.51	50.74 $\pm$ 0.66	35.00 $\pm$ 0.52
		LFT $T_1 - T_2$ (our run)	46.03 $\pm$ 0.60	31.50 $\pm$ 0.49	49.29 $\pm$ 0.65	36.34 $\pm$ 0.59
		LFT EvoGrad	47.39 $\pm$ 0.61	32.51 $\pm$ 0.56	50.70 $\pm$ 0.66	36.00 $\pm$ 0.56
5-way 5-shot	ResNet34	-	45.61 $\pm$ 0.59	29.54 $\pm$ 0.46	48.87 $\pm$ 0.65	35.03 $\pm$ 0.54
		FT	45.15 $\pm$ 0.59	30.28 $\pm$ 0.44	49.96 $\pm$ 0.66	35.69 $\pm$ 0.54
		LFT EvoGrad	45.97 $\pm$ 0.60	33.21 $\pm$ 0.54	50.76 $\pm$ 0.67	38.23 $\pm$ 0.58
5-way 1-shot	ResNet10	-	62.13 $\pm$ 0.74	40.64 $\pm$ 0.54	64.34 $\pm$ 0.57	46.29 $\pm$ 0.56
		FT	63.64 $\pm$ 0.77	42.24 $\pm$ 0.57	65.42 $\pm$ 0.58	47.81 $\pm$ 0.51
		LFT $T_1 - T_2$	64.99 $\pm$ 0.54	43.44 $\pm$ 0.59	67.35 $\pm$ 0.54	50.39 $\pm$ 0.52
		LFT $T_1 - T_2$ (our run)	65.94 $\pm$ 0.56	43.88 $\pm$ 0.56	65.57 $\pm$ 0.57	51.43 $\pm$ 0.55
		LFT EvoGrad	64.63 $\pm$ 0.56	42.64 $\pm$ 0.58	66.54 $\pm$ 0.57	52.92 $\pm$ 0.57
5-way 5-shot	ResNet34	-	63.33 $\pm$ 0.59	40.50 $\pm$ 0.55	64.94 $\pm$ 0.56	50.20 $\pm$ 0.55
		FT	62.48 $\pm$ 0.56	41.06 $\pm$ 0.52	64.39 $\pm$ 0.57	50.08 $\pm$ 0.55
		LFT EvoGrad	66.40 $\pm$ 0.56	44.25 $\pm$ 0.55	67.23 $\pm$ 0.56	52.47 $\pm$ 0.56

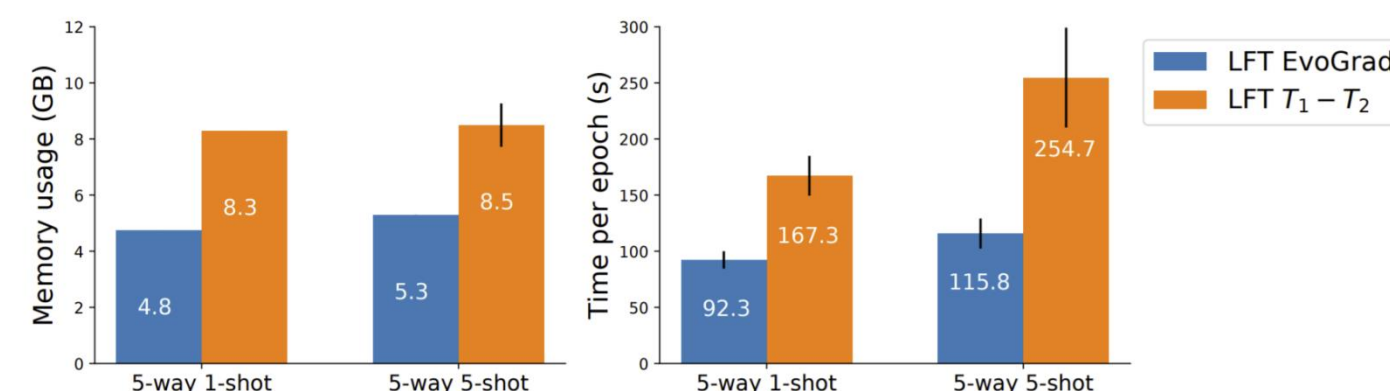


Figure 3: Cross-domain few-shot learning with LFT: analysis of memory and time efficiency of EvoGrad vs standard second-order $T_1 - T_2$ approach. Mean and standard deviation reported across experiments with different test datasets. EvoGrad is significantly more efficient in terms of both memory usage and time per epoch.

Noisy label learning with MetaWeightNet

Table 4: Test accuracies (%) for Meta-Weight-Net label noise experiments with ResNet-32 – means and standard deviations across 5 repetitions for the original second-order algorithm vs EvoGrad. EvoGrad is able to match or even exceed the accuracies obtained by the original MWN approach.

Dataset	Noise rate	Baseline	MWN $T_1 - T_2$	MWN $T_1 - T_2$ (our run)	MWN EvoGrad
CIFAR-10	0%	92.89 $\pm$ 0.32	92.04 $\pm$ 0.15	91.10 $\pm$ 0.19	92.02 $\pm$ 0.31
	20%	76.83 $\pm$ 2.30	90.33 $\pm$ 0.61	89.31 $\pm$ 0.40	89.86 $\pm$ 0.64
	40%	70.77 $\pm$ 2.31	87.54 $\pm$ 0.23	85.90 $\pm$ 0.45	87.74 $\pm$ 0.54
CIFAR-100	0%	70.50 $\pm$ 0.12	70.11 $\pm$ 0.33	68.42 $\pm$ 0.36	69.16 $\pm$ 0.49
	20%	50.86 $\pm$ 0.27	64.22 $\pm$ 0.28	63.43 $\pm$ 0.43	64.05 $\pm$ 0.63
	40%	43.01 $\pm$ 1.16	58.64 $\pm$ 0.47	56.54 $\pm$ 0.90	57.44 $\pm$ 1.25

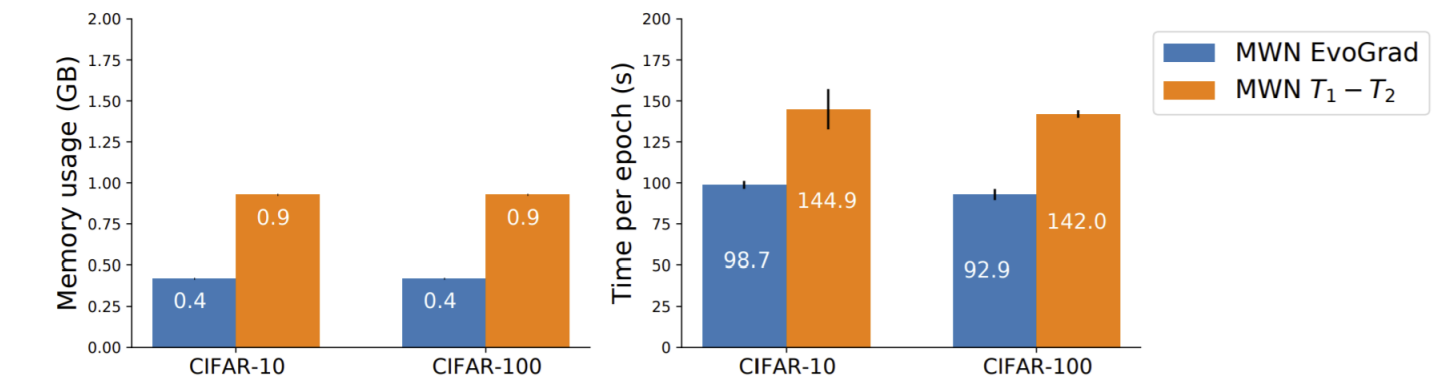


Figure 4: Analysis of memory and time cost of MWN EvoGrad vs the original second-order MWN, showing significant efficiency improvements by EvoGrad. Mean and standard deviation is reported across 5 repetitions of 40% label noise problem.

Scalability analysis

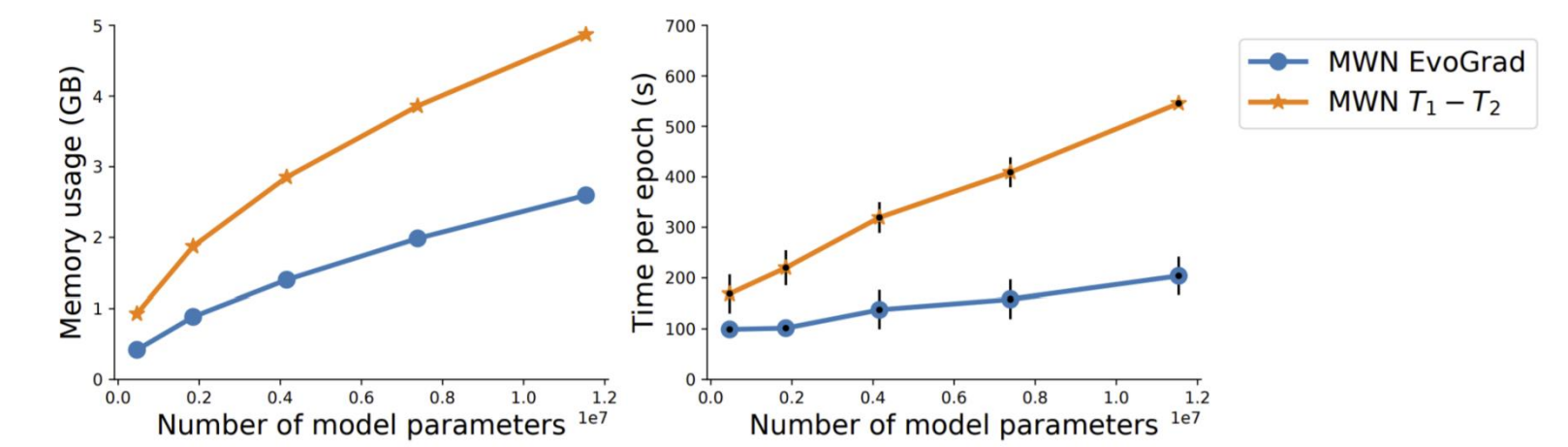


Figure 5: Memory and time scaling of MWN EvoGrad vs original second-order Meta-Weight-Net. Efficiency margins of EvoGrad are larger for larger models.

Low-resource cross-lingual learning with MetaXL

Table 5: Test F1 score in % for named entity recognition task. English source language. The first two rows are taken from the MetaXL paper, while our own runs are in the following rows. EvoGrad clearly matches and even surpasses the performance of $T_1 - T_2$ baseline. Joint-training (JT) represents a simple non-meta-learning baseline approach.

Method	qu	cdo	ilo	xmf	mhr	mi	tk	gn	Average
JT	66.10	55.83	80.77	69.32	71.11	82.29	61.61	65.44	69.06
MetaXL $T_1 - T_2$	68.67	55.97	77.57	73.73	68.16	88.56	66.99	69.37	71.13
JT (our run)	59.75	49.19	79.43	68.85	68.42	89.94	61.90	69.44	68.37
MetaXL $T_1 - T_2$ (our run)	65.29	56.33	76.50	67.24	71.17	89.41	66.67	64.11	69.59
MetaXL EvoGrad	71.00	57.02	85.99	70.40	65.45	88.12	66.97	70.91	71.98

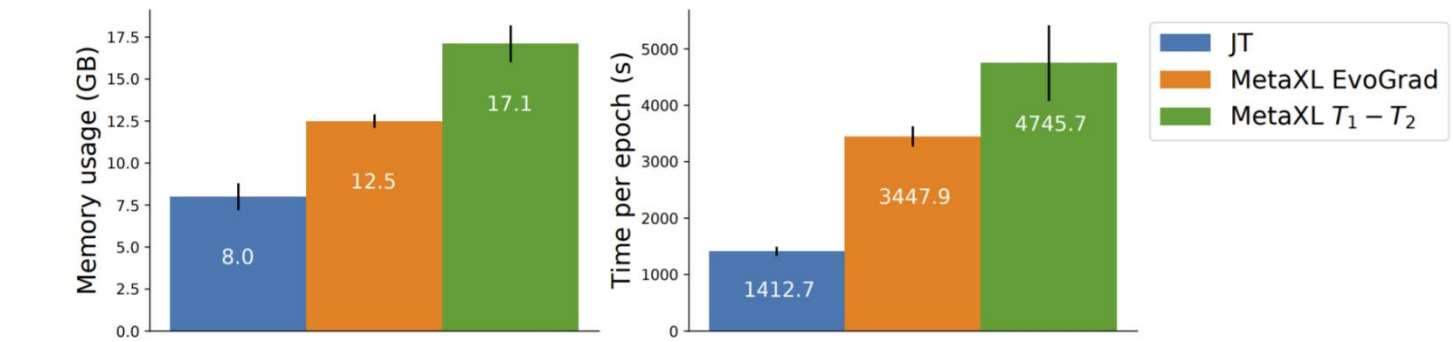


Figure 6: Analysis of memory and time cost of MetaXL EvoGrad vs the original second-order MetaXL, in the context of a simple joint-training (JT) baseline. EvoGrad consumes significantly less memory than $T_1 - T_2$ and is faster. Mean and standard deviation is calculated over the 8 different target languages.

References

- Luketina et al. (2016). “Scalable gradient-based tuning of continuous regularization hyperparameters” ($T_1 - T_2$). In: ICML.
- Tseng et al. (2020). “Cross-domain few-shot classification via learned feature-wise transformation”. In: ICLR.
- Shu et al. (2019). “Meta-Weight-Net: learning an explicit mapping for sample weighting”. In: NeurIPS.
- Xia et al. (2021). “MetaXL: meta representation transformation for low-resource cross-lingual learning”. In: NAACL.

